Wafa وفاء

Keep your word.

A small, deployed **request** → **review** → **decision** app for interest-free (قرض حسن · qard hasan) loans between friends. A faithful, shared record of an agreement — not a place money moves.

WHAT WAS ASKED

One approval flow, two kinds of users

The prompt: companies have no clean way to ask for something that needs approval — it is scattered across email and chat. Build one **request → review → decision** flow, with a person who requests and a person who approves.

MY INTERPRETATION

Mal is an AI-native, **Sharia-compliant fintech**. So I chose an approval flow that is native to that world: a friend **requests a goodwill loan**, a friend **approves, counters, or declines**, and it is tracked to repaid. Same primitive, a domain that actually resonates.

WHY IT FITS THE ASSIGNMENT

Two clear roles, **per loan** (anyone can be borrower or lender). A real decision with a real counter-offer. A genuine pain — chasing a friend for money is awkward — solved by a written record, not a feature pile.

WHAT WAFA IS

A shared ledger of a promise

Wafa records *qard hasan* — interest-free, goodwill loans between people who already trust each other. It is a faithful, auditable log of an agreement: requested, approved or countered, transferred, repaid, confirmed.

Record, not a rail

No money moves through Wafa. A deliberate scoping choice that keeps it genuinely shippable and well clear of payments and regulatory scope — while still solving the real pain: the tracking.

Interest-free by design

There is deliberately **no interest or markup column anywhere** in the schema. The qard-hasan identity is enforced by absence, not by a setting someone could flip.

Status before action

Every screen first answers **"where does this stand, and is the ball in my court?"** Colour carries state, so you read a loan before you decide what to do with it.

interest-free · qard hasan — surfaced calmly and often

REQUEST → REVIEW → DECISION → SETTLED

One lifecycle, fully tracked

Request
Borrower asks, in plain words

→

Review
Approve · counter · decline

→

Active
Agreed & outstanding

→

Transferred
Borrower marks it sent

→

Settled
Lender confirms received

STATUS SPEAKS IN COLOUR

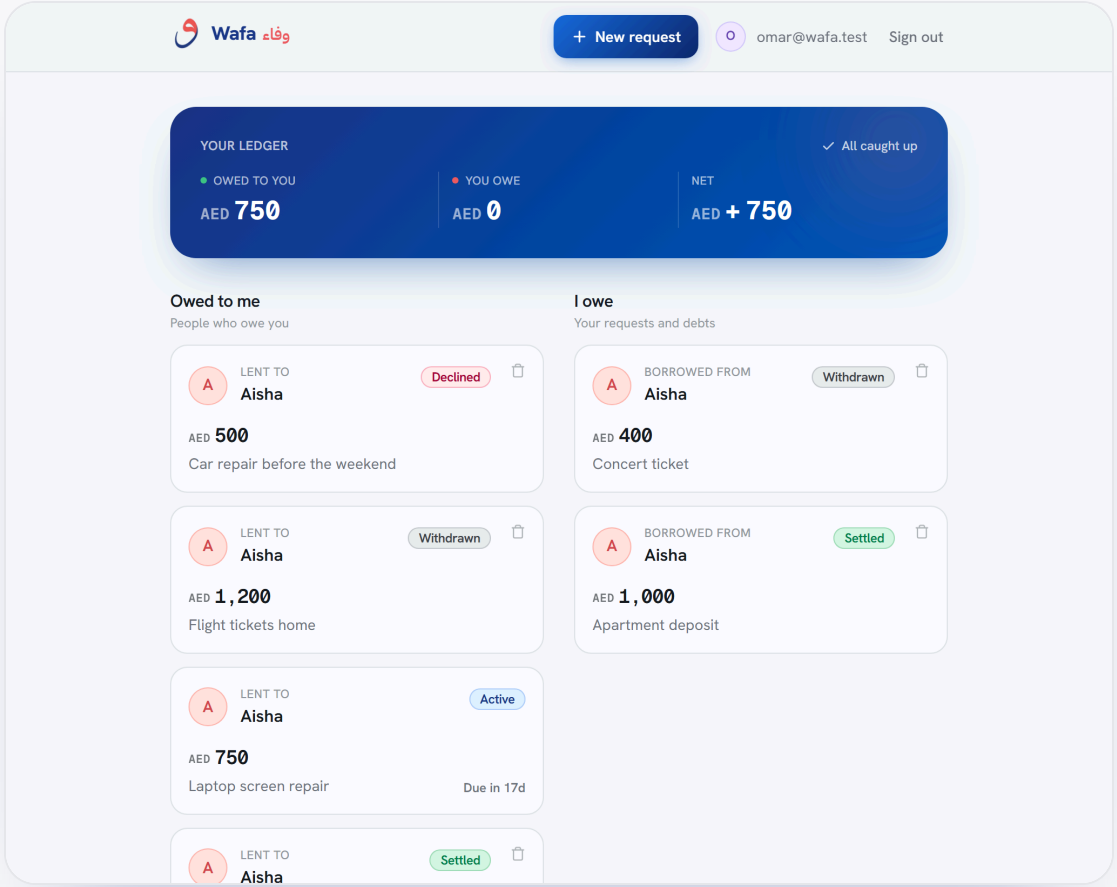
- Pending
- Countered — your move
- Active
- Awaiting confirmation
- Settled
- Declined
- Withdrawn

TWO STRUCTURAL INVARIANTS

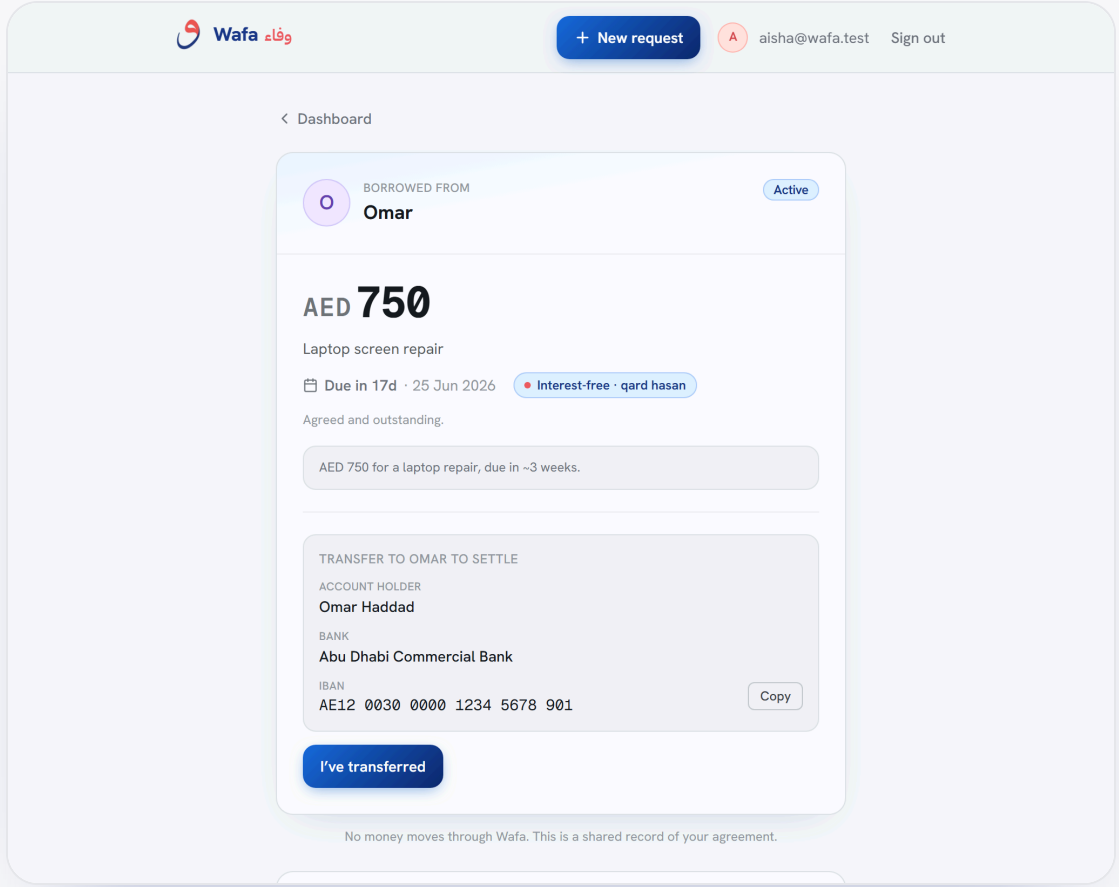
Single bounce: the one allowed counter-offer is folded into the row, so a second counter is structurally unreachable. **Every transition** writes an immutable row to an audit timeline both parties can see.

TWO COLUMNS: OWED TO ME · I OWE

The ledger, and a loan in detail



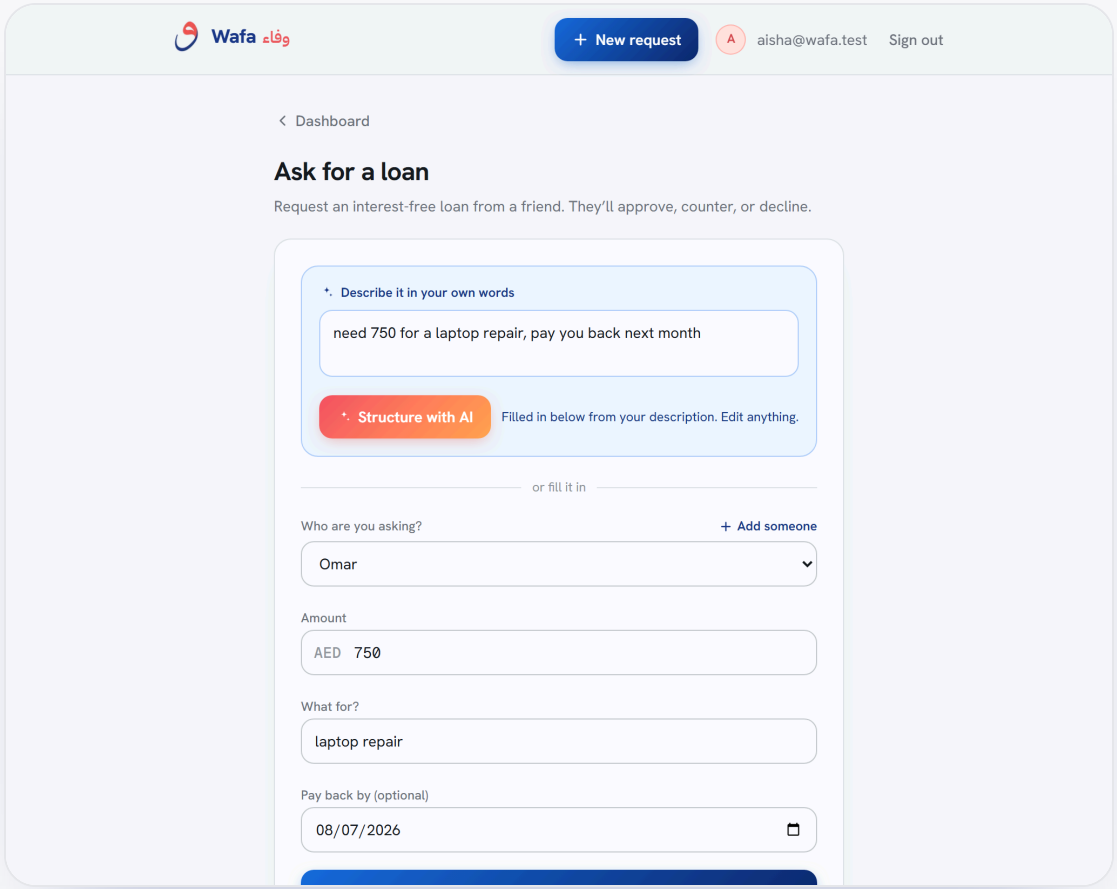
Dashboard. The pending inbox and the two-column ledger.



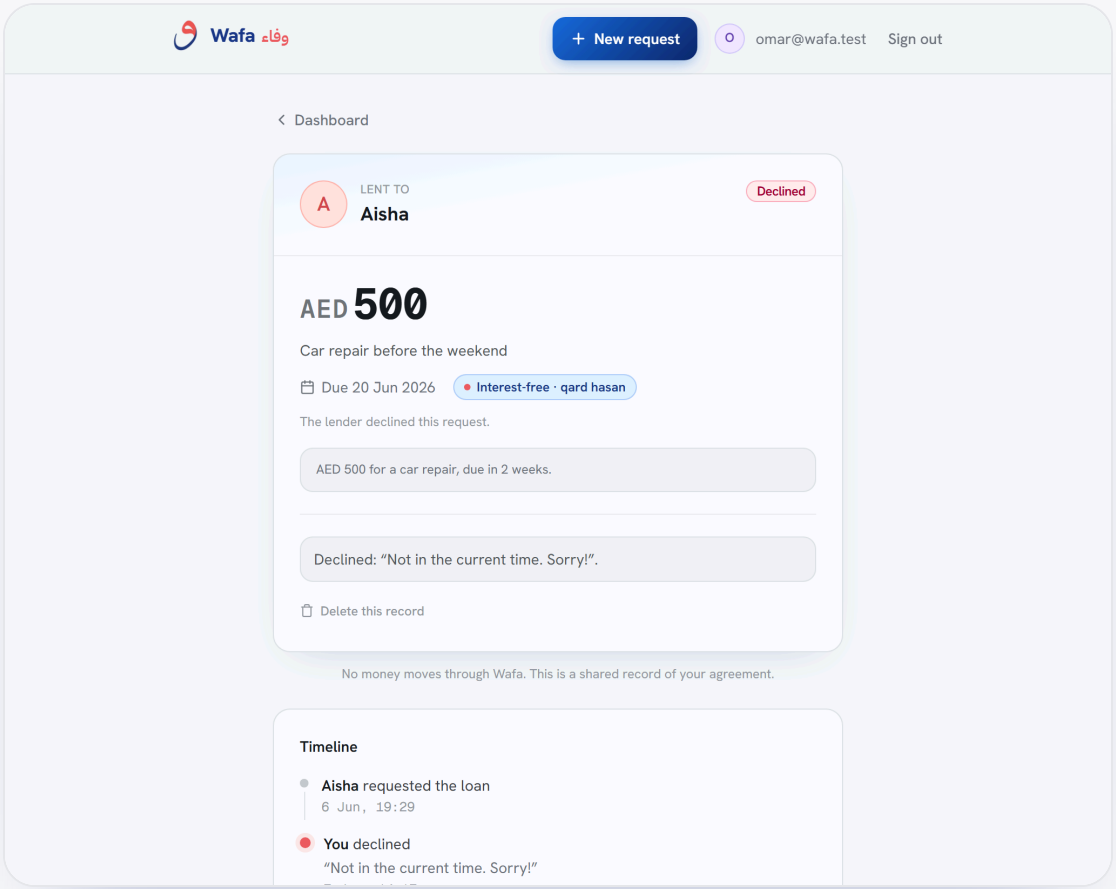
Active loan. Repayment details to settle, plus the shared timeline.

DESCRIBE IT IN PLAIN WORDS

AI-assisted request & the pending review



New request. Free text → Claude structures amount, reason, due date. Form stays editable.



Pending review. The lender's view: approve, counter, or decline.

THE INTERESTING PART IS THE DATA LAYER

Security that does not trust the client

- **Every state transition is a Postgres `SECURITY DEFINER` RPC.** It asserts the caller's role and the exact current status, then does the UPDATE and the audit INSERT atomically.
- **The `loans` table has no write policy at all.** Direct inserts/updates/deletes are denied — the RPCs are the only write path, sidestepping the classic mis-scoped-RLS bug.
- **RLS scopes every read** to the loan's two parties.
- **Payment details are column-protected.** Bank fields (IBAN, holder, SWIFT…) are unreadable directly; the lender's reach the borrower only via a definer RPC, **borrower-only and active-only**.
- **Single bounce is structural**, and **interest-free is enforced by absence** — no markup column exists.
- **An in-DB test harness verifies the guarantees** — wrong-party, wrong-status, RLS isolation, direct-write denial, IBAN rules.

11 / 11
in-DB security guarantees pass

0
error-level advisor lints

1
deliberate write boundary (the RPCs)

LIGHT, HONEST, NEVER BLOCKING

Two AI touches — text and vision

Structure a request from plain words

“Need 400 for a car repair, pay you back in two weeks.” A server-side Claude call turns it into amount / reason / due date, plus a plain-terms interest-free summary, and **pre-fills a fully editable form**.

Read payment details from a photo

Snap a bank document in Settings; the same model reads it into the payment fields — again only pre-filling. **The image is used for that one call and never stored.**

Deliberately non-blocking. Each route has an **8-second timeout** and **always returns HTTP 200** — a slow, missing, or erroring model never blocks anything. The UI simply falls back to the manual form. Claude Haiku 4.5, via OpenRouter, server-side only.

THE STACK

Shipped on the required tools

Next.js 16 · Vercel

App Router, TypeScript, React Server Components + Server Actions. Every route paints an on-brand skeleton the instant it is clicked.

Supabase

Postgres, Auth, and Row-Level Security. Seven migrations, a seed across every status, and an in-DB test harness.

Claude · Claude Code

Claude Haiku 4.5 via OpenRouter for the two light AI touches. The app itself was built with Claude Code.

FRONT OF HOUSE

Tailwind CSS v4 with OKLCH tokens · three typefaces, each with a job (Hanken Grotesk, IBM Plex Sans Arabic, Geist Mono) · reduced-motion-safe throughout · mobile-first, excellent at 390px.

VERIFICATION

Standalone scripts for auth + RLS + IBAN protection, the full lifecycle over PostgREST, the AI structuring, and a Playwright drive-through that doubles as the screenshot generator.

WARM AND TRUSTWORTHY, NOT A BANKING CONSOLE

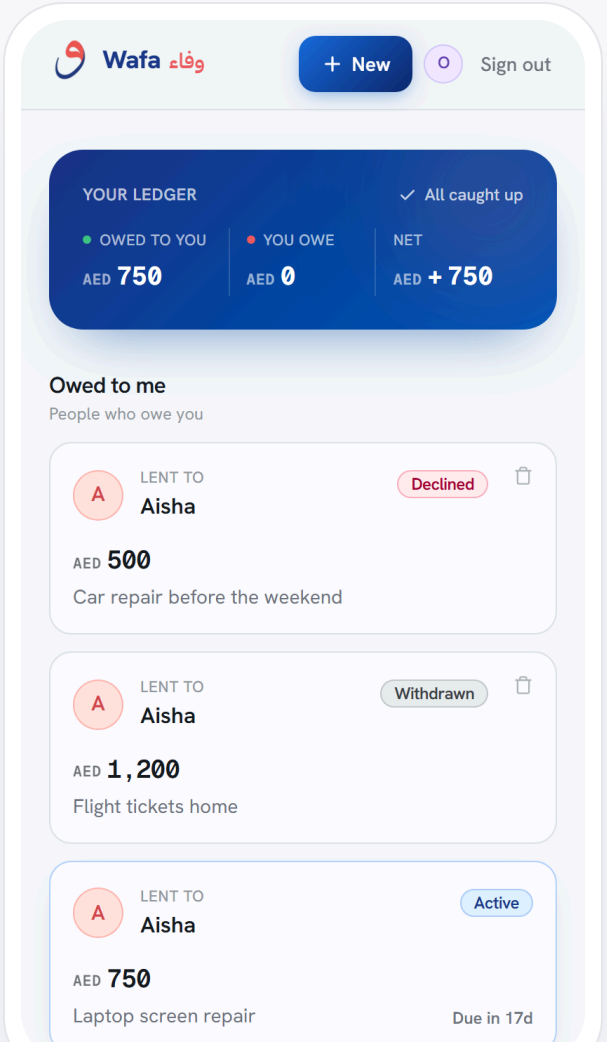
Bilingual by identity, mobile by default

A fresh, bright canvas

Deep-blue + coral brand DNA, with joyful accents (teal, amber, violet, mint) spent in committed colour moments — a gradient ledger band, mesh hero, gradient CTAs — never spread evenly.

Money lines up like a ledger

Geist Mono with `tabular-nums` for every figure and timestamp. The وفاء wordmark is set in IBM Plex Sans Arabic, `dir="rtl"`.



WHAT I DELIBERATELY LEFT OUT — AND WHY

Knowing what not to build

The brief asked for something **small**. A take-home is partly a test of scoping judgment, so two tempting features were consciously deferred:

Installment repayment

"5,000, paid 2,500 + 2,500." A natural pattern, but it lives entirely in the **post-decision tail** — furthest from the assessed flow — and adds real surface area. Named on the roadmap as a deliberate next step.

Public sign-up

Wafa is inherently **two-sided** — a fresh sign-up lands on an empty dashboard with no counterparty. Curated demo accounts that already know each other show the product far better. Sign-up waits for a real invite story.

The point: each decision keeps the one flow tight and well-built, rather than spreading effort thin across a longer feature list.

WHERE IT GOES NEXT

Try it, and the road ahead

LIVE DEMO

Open **wafa.7mxd.me** and sign in with one tap, or use any account below.

Password for all accounts: `Wafa-demo-1`

- `aisha@wafa.test`
- `omar@wafa.test`
- `layla@wafa.test`
- `yusuf@wafa.test`

NATURAL NEXT STEPS

- Due-date reminders & **installment repayment**
- **Full Arabic localization** and RTL layout (the identity is already bilingual)
- A **shareable invite link** for friends not yet on Wafa, & lent-vs-borrowed stats
- On the same request→confirm primitive: **gam'iya** savings circles, fair cost splits, sadaqah pools

وفاء — keep your word.